

AER8500

Informatique embarquée de l'avionique

Rapport Final de Mini-Projet

Régulation de l'altitude d'un avion

LETT Bastien & POLZIN Laurent

Hiver 2026

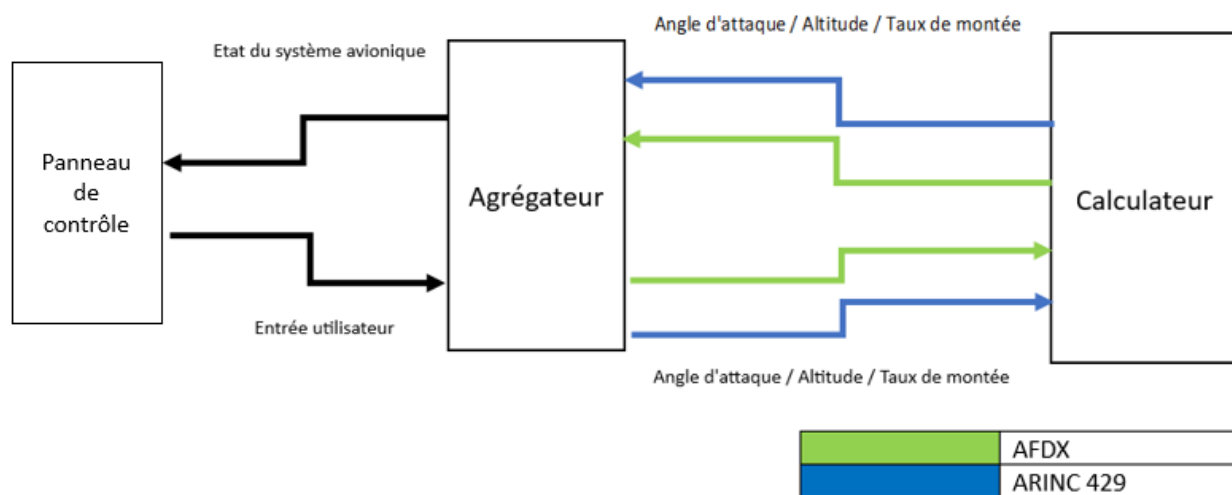
1. Division du projet en tâches

Le projet a été découpé en trois modules fonctionnels indépendants, développés en C, reflétant l'architecture avionique décrite dans l'énoncé : le panneau de contrôle (panel), l'agrégateur et le calculateur. Les tâches ont été réparties comme suit :

Tâche	Description	Responsable
Module Panneau (panel.c/.h)	Gestion des entrées utilisateur, validation, affichage	
Module Calculateur (calculateur.c/.h)	Algorithme de régulation d'altitude, formules de taux de montée	
Module Agrégateur (agregateur.c/.h)	Orchestration entre le panneau et le calculateur	
Tests unitaires	Tests panneau, calculateur, cas limites	
Interface graphique Windows (winui.c)	Cockpit simulé : saisies, instruments, graphe d'altitude	
Communication AFDX	Redondance et transmission rapide des données sensibles	
Communication ARINC 429	Transmission des mots binaires/BCD (labels 001, 002, 003)	

2. Algorithme de la fonction avionique

2.1 Architecture générale



L'architecture suit le diagramme d'interaction proposé plus tôt dans notre rendu précédent. Le panneau recueille les entrées utilisateur et les transmet à l'agrégateur, qui pilote le calculateur. Le calculateur retourne un nouvel état système que l'agrégateur injecte dans le panneau pour mise à jour de l'affichage. Le cycle s'exécute toutes les 50 ms (interface graphique) ou en boucle discrète de 10 s (simulation console).

2. Etats du mode avionique

La fonction avionique repose sur trois états conformes à l'énoncé :

- AU_SOL : Altitude courante = 0 ft et altitude désirée = 0 ft. Les taux de montée et vitesse sont nuls.
- CHANGEMENT_ALT : altitude courante et altitude désirée différents. Le calculateur applique la formule de taux de montée et met à jour l'altitude à chaque pas de temps.
- VOL_CROISIÈRE : altitude désirée atteinte ou altitude maximale de 40 000 ft atteinte. Le taux de montée est mis à 0 et l'altitude et la puissance restent constants.

La transition entre l'état AU_SOL et l'état CHANGEMENT_ALT est déclenchée dès que l'utilisateur entre une altitude désirée non nulle via `panel_set_altitude_desiree()`. Le mode est mis à jour directement dans le panneau pour assurer une réactivité immédiate.

3. Formule du taux de montée

Le taux de montée est calculé de la manière suivante :

- En VOL_CROISIÈRE ou AU_SOL : taux = 0 m/min.
- En CHANGEMENT_ALT : taux de base = $100 \text{ m/min} \times (\text{puissance_moteur} / 10)$. Avec une puissance nominale de 50 %, on obtient 500 m/min, dans la limite des 800 m/min spécifiés.
- A l'approche de l'altitude désirée, le taux de montée s'adapte pour une transition plus douce.

4. Encodage conforme ARINC

Nous avons choisi de créer une structure « mot arinc 429 », qui sépare nettement les champs, par exemple le champ « parité » ou encore « data », mais possède également la trame brute, de 32 bits.

L'implémentation est contre-intuitive : le stockage effectif de la trame en bit est inversée par rapport à notre lecture humaine. Le bit de poids faible est « à droite », mais la traduction du champ data, par exemple, se fait « par la gauche », avec le champ « label » qui est lui aussi inversé, on s'y perd rapidement si on ne se met pas d'accord immédiatement sur la manière de stocker les données.

La trame brute est le champ en qui nous devons faire confiance. Effectivement, selon l'encodage (BNR ou BCD), les champs ne seront pas les mêmes : le bit 29 désigne le signe de la valeur que nous encodons en BNR, mais est le MSB en BCD.

À chaque fonction d'encodage est associée son inverse, ainsi, on peut encoder en BNR et décoder le BNR, mais également créer un mot A429 avec « `t_a429_word get_A429_word(uint8_t label, float value, int etat)` » (où « état est ignoré si label ne vaut pas 1) avant d'en obtenir la valeur encodée avec « `float get_value_from_a429(t_a429_word w)` ».

5. Encodage conforme AFDX

De la même manière, nous avons créé une structure de données « trame_afdx ». Elle délimite clairement chacun des 11 champs : « preamble », « sfd », adresse mac de la source et de la destination (on a choisi @Mac agregateur = 1 et @Mac calculateur = 2), le type du paquet est IPv4. Nous remplissons les en-tête IP et UDP, calculons les checksums. Les numéros de séquence des paquets sont cycliques, de 0 à 255. Entre chaque trame AFDX, il y a un 12 octets à 0. On ajoute également un champ « FS » pour connaître l'état des données transmises.

3. Identification des tâches critiques

Les tâches critiques ont été identifiées selon leur impact sur la sûreté de fonctionnement et la conformité à l'énoncé :

Tâche critique	Risque associé	Niveau
Validation des entrées utilisateur	Valeurs hors intervalle transmises au calculateur	ÉLEVÉ
Gestion de la machine à états	Transition incorrecte entre les états	ÉLEVÉ
Formule du taux de montée	Dépassement de 800 m/min ou oscillations à l'approche de la cible	ÉLEVÉ
Clamp d'altitude [0, 40 000 ft]	Altitude négative ou supérieure au plafond autorisé	MOYEN
Détection du cas de décrochage	Chute libre non gérée	MOYEN
Implémentation AFDX/A429 (future)	Intégrité des données non garantie sans protocole formalisé	MOYEN

4. Plan de contingence

Les mesures suivantes ont été prévues ou mises en place pour faire face aux risques identifiés :

- Validation stricte en entrée : chaque setter du panneau (`panel_set_altitude_desiree`, `panel_set_taux_montee`, `panel_set_angle`) vérifie les bornes et enregistre un message d'erreur explicite via `panel_get_last_error()`. Toute valeur invalide est rejetée sans modifier l'état du système.
- Limitation dans le calculateur : l'altitude résultante est systématiquement bornée à [0, 40 000 ft]. Le taux de montée est limité à 800 m/min. En cas de dépassement, le système bascule automatiquement en VOL_CROISIÈRE.
- Snap à l'altitude cible : lorsque l'écart résiduel est inférieur à 50 ft, l'altitude est directement alignée sur la cible et le mode VOL_CROISIÈRE activé, évitant les oscillations autour de la consigne.
- Architecture modulaire : chaque module (panneau, calculateur, agrégateur) est compilable et testable indépendamment. La séparation des couches facilite dans notre cas le remplacement de l'agrégateur par une vraie couche AFDX/A429 sans toucher au reste.
- Tests unitaires automatisés (`testsPanneau.c`, `testsCalculateur.c`) : couvrent les cas nominaux et les cas limites (altitude invalide, taux hors borne, clamp à 40 000 ft).
- Interface graphique comme outil de validation : la fenêtre Windows (`winui.c`) affiche en temps réel le graphe d'altitude, le mode avionique, l'indicateur d'angle d'attaque et les instruments, permettant de détecter visuellement tout comportement anormal.

5. Critique du travail pratique

Le projet nous a permis de mettre en pratique les concepts d'informatique embarquée dans un contexte avionique réaliste. Plusieurs points méritent d'être soulignés :

Points forts

- La séparation claire en modules (panneau, agrégateur, calculateur) facilite la maintenance et l'évolution du code, conformément aux bonnes pratiques du génie logiciel embarqué.
- La validation systématique des entrées et la gestion explicite des erreurs renforcent la robustesse du système, qualité essentielle en avionique.
- L'interface graphique Windows offre une démo visuellement convaincante et facilite la détection de bugs comportementaux difficiles à voir en mode texte.

Limites et améliorations possibles

- La puissance moteur est actuellement une valeur fixe (50 %). Un modèle dynamique liant puissance, angle d'attaque et taux de montée aurait rendu la simulation plus fidèle à la réalité.
- Le cas de décrochage (angle > 15°) est mentionné dans l'énoncé mais n'est pas encore géré dans l'algorithme : une transition vers un état de sécurité (ex. réduction de l'angle d'attaque imposée) devrait être ajoutée.
- Le projet gagnerait à être testé sur une cible embarquée réelle (Raspberry Pi ou plateforme RTOS) pour mettre en évidence les performances temporelles et la consommation mémoire.